historical_results 테이블 확인 가이드

cheongyak-platform 팀 프로젝트 · DB 데이터 보는 법 + 컬럼 설명 + 청약홈 API로 가져온 방법

> 이 문서는 세 부분으로 구성됩니다 — ① VS Code / DBeaver로 `historical_results` 테이블 데이터를 조회 화면까지 들어가는 경로, ② 각 컬럼(열)이 실제로 어떤 값을 의미하는지 매칭표, ③ 이 데이터를 청약홈 공공데이터 API로 어떻게 수집했는지 파이프라인 설명.

> **먼저 알아둘 것** — 이 테이블 하나에 성격이 다른 **3가지 종류의 행**(경쟁률 / 당첨가점 / 특별공급 신청현황)이 함께 저장돼 있습니다. 같은 컬럼이라도 행 종류에 따라 뜻이 달라지는 경우가 있어서, 2번 섹션에서 종류별로 나눠 설명합니다.

1 데이터 조회 화면 들어가는 법

VS Code PostgreSQL 확장으로 보기

1. **명령어 팔레트 열기** — `Ctrl+Shift+P`
2. **"보기: PostgreSQL 표시"** (View: Show PostgreSQL) 입력 후 실행 → 왼쪽 사이드바에 연결 트리가 열림
3. 트리에서 **연결(cheongyak)** → **Databases** → `cheongyak` → **Schemas** → `public` → **Tables** 순서로 펼치기
4. `historical_results` 테이블을 찾아 **마우스 우클릭** → 데이터 조회 메뉴("Select Top 1000 Rows" 등) 클릭
5. 화면 상단 **"결과"** 탭에 `public.historical_results` 제목으로 표 형태 결과가 뜨면 정상

DBeaver로 보기

1. 왼쪽 **Database Navigator**에서 `cheongyak` 연결 → `Databases` → `cheongyak` → `Schemas` → `public` → `Tables` 순서로 펼치기
2. `historical_results` **더블클릭** → 상단 **"Data"** 탭 클릭 → 표 형태로 데이터가 뜸
3. SQL로 직접 조회하려면 상단 **SQL Editor** 아이콘 클릭 → `SELECT * FROM historical_results LIMIT 100;` 실행 (Ctrl+Enter)

> 화면이 안 뜨면: DB가 켜져 있는지(`docker compose up -d db`) 먼저 확인하고, 그래도 안 되면 연결이 끊긴 것이니 연결을 다시 클릭해서 재연결하세요.

2 이 테이블에 담긴 3가지 행 종류

같은 테이블 안에서 아래처럼 구분됩니다. `category_label` 값의 형식과, 어떤 값 계열 컬럼이 채워져 있는지로 구별할 수 있습니다.

종류	대상 house_type	category_label 형식	채워지는 컬럼
경쟁률	APT / 오피스텔·도시형생활주택 / 무순위·잔여세대 / 공공지원민간임대 / 임의공급 (5개 전부)	유형마다 다름 — 예: "해당지역 1순위" (APT), "전체"(오피스텔), "사후"/"사전" (무순위), "일반공급"(공공지원민간임대), 없음(임의공급)	`supply_count`, `apply_count`, `competition_rate`, `is_undersubscribed`, `raw_cmpet_rate`
당첨가점	APT만	"해당지역" 또는 "기타지역" (지역만, 순위 구분 없음)	`avg_score`, `cutoff_score`, `top_score`
특별공급 신청현황	APT만	"{특공유형} - {지역}" 형식 — 예: "생애최초 - 해당지역", "신혼부부 - 기타경기"	`supply_count`, `apply_count`, `competition_rate` (자체계산), `is_undersubscribed` (자체계산)

SQL로 종류를 구분해서 세어보려면:

```
SELECT
  CASE
    WHEN category_label LIKE '% - %' THEN '특별공급'
    WHEN avg_score IS NOT NULL OR cutoff_score IS NOT NULL OR top_score IS NOT NULL THEN '당첨가점'
    ELSE '경쟁률'
  END AS 데이터종류,
  COUNT(*) AS 행수
FROM historical_results
GROUP BY 1
ORDER BY 1;
```

3 컬럼명 매칭표

API 원본 청약홈 공공데이터 API 응답 필드를 그대로 옮긴 값 가공 여러 필드를 조합했거나 직접 계산/파싱해서 만든 값

공통 컬럼 (3가지 종류 모두 해당)

컬럼명	원본 API 필드	의미
`id`	–	가공 DB에서 행마다 자동으로 매기는 일련번호. API와 무관하게 우리 DB 내부에서만 쓰는 값.
`external_id`	PBLANC_NO	API 원본 청약홈 **공고번호**. `listings.external_id`와 같은 값이라 두 테이블을 이 컬럼으로 조인할 수 있음.
`unit_type`	HOUSE_TY	API 원본 **주택형(평형) 코드**. `listings.unit_type`과 같은 체계.
`house_type`	–	가공 어느 경쟁률 API로 가져온 행인지 코드에서 직접 지정한 값 (APT/오피스텔·도시형생활주택/무순위·잔여세대/공공지원민간임대/임의공급). 당첨가점·특별공급 행은 전부 "APT".
`category_label`	여러 필드 조합	가공 행의 세부 구분 라벨. 원본 API가 지역/순위/특공유형 등을 여러 필드(컬럼)로 나눠 주는 걸, 사람이 읽기 쉬운 문자열 하나로 합쳐서 저장한 것. 종류별 조합 방식은 2번 섹

션 표 참고.

경쟁률 행 전용 컬럼

컬럼명	원본 API 필드	의미
`supply_count`	SUPLY_HSHLDCO	API 원본 그 구분(지역×순위 등) 기준 **공급세대수**.
`apply_count`	REQ_CNT	API 원본 **신청자수**.
`competition_rate`	CMPET_RATE	API 원본 **경쟁률**. CMPET_RATE가 "4.40"처럼 숫자로 된 경우만 채워짐. "-"(데이터없음)나 "(△209)"(미달) 표기는 숫자로 못 바꾸니 NULL로 저장됨.
`is_undersubscribed`	CMPET_RATE (파싱)	가공 CMPET_RATE가 "(△209)"처럼 **미달(△) 표기**인 경우 true. 그 외에는 false.
`raw_cmpet_rate`	CMPET_RATE	API 원본 API가 준 **원본 문자열 그대로** (숫자/"-"/미달표기 다 포함). 검증·디버깅용으로 남겨둔 값.

당첨가점 행 전용 컬럼 (APT만)

컬럼명	원본 API 필드	의미
`avg_score`	AVRG_SCORE	API 원본 **평균 당첨가점**. 값이 없으면("-") NULL.
`cutoff_score`	LWET_SCORE	API 원본 **최저 당첨가점(커트라인)** — 실제로 당첨된 사람 중 가장 낮은 점수. "내가 이 점수면 될까?" 판단할 때 기준이 되는 값.
`top_score`	TOP_SCORE	API 원본 **최고 당첨가점**.

특별공급 신청현황 행 전용 컬럼 (APT만)

컬럼명	원본 API 필드	의미
`supply_count`	각 특공유형의 `*_HSHLDCO`	API 원본 그 **특별공급 유형의 배정세대수** (지역 구분 없이 하나의 값 — 해당지역/기타경기/기타지역 3개 행에 동일하게 반복 저장됨).
`apply_count`	지역×유형의 `*_CNT`	API 원본 그 **지역·유형 조합의 접수건수**.
`competition_rate`	— (자체 계산)	가공 이 API는 경쟁률을 직접 안 줘서, `apply_count ÷ supply_count`로 우리가 계산해서 채운 값 (소수점 2자리 반올림).
`is_undersubscribed`	— (자체 계산)	가공 `apply_count < supply_count`이면 true로 직접 계산.
`raw_cmpet_rate`	—	해당없음 — 이 API는 경쟁률 텍스트를 안 줘서 항상 NULL.

특별공급 유형 7종: 기관추천 · 다자녀 · 노부모부양 · 신혼부부 · 생애최초 · 신생아 · 청년
지역 구분 3단계: 해당지역(그 공고가 있는 시/군 거주자 우선) → 기타경기(수도권 공고 한정, 경기도 내 다른 지역) → 기타지역(그 외 전국)

4 청약홈 API로 가져온 방법

`listings`는 "분양정보 조회 서비스"(공공데이터포털 15098547) API로 가져왔는데, `historical_results`는 별도 API 상품인 **"청약접수 경쟁률 및 특별공급 신청현황 조회 서비스"**(공공데이터포털 15098905)에서 가져왔습니다. 인증키(APPLYHOME_API_KEY)는 같은 걸 씁니다.

```
기본 주소: https://api.odcloud.kr/api/ApplyhomeInfoCmpetRtSvc/v1
```

① 왜 "발표 완료된 공고"만 가져오는가

경쟁률·당첨가점·특별공급 신청현황은 **당첨자 발표가 끝난 공고**에만 실제 값이 존재합니다. 그래서 `listings`를 가져올 때 이미 받아둔 `PRZWNER_PRESNATN_DE`(당첨자 발표일)가 오늘 날짜보다 지난 공고만 걸러내서, 그 공고들에 대해서만 아래 API들을 호출합니다. (2026-09-18 기준 공공지원민간임대·임의공급은 발표 완료된 공고가 아직 0건이라 이 두 유형은 경쟁률 데이터가 비어있는 상태 — 버그 아니고, 데이터가 아직 안 생긴 것.)

② 호출 방식: 공고 하나당 API 1번씩 (N+1 패턴)

이 API들은 날짜 범위로 여러 공고를 한 번에 주는 게 아니라, `cond[PBLANC_NO::EQ]` 파라미터로 **공고번호 하나씩만** 조회할 수 있습니다. 그래서 발표 완료된 공고 목록을 먼저 뽑아둔 다음, 그 목록을 하나씩 돌면서 API를 반복 호출하는 방식으로 만들었습니다 (공고 221개면 API 호출도 221번).

③ 사용한 엔드포인트 (오퍼레이션) 목록

`getAPTLttotPblancCmpet` · APT 경쟁률 — house_type="APT" 경쟁률 행

`getUrbtyOfctlLttotPblancCmpet` · 오피스텔·도시형생활주택 경쟁률

`getRemndrLttotPblancCmpet` · 무순위·잔여세대 경쟁률

`getPblPvtRentLttotPblancCmpet` · 공공지원민간임대 경쟁률

getOPTLttotPblancCmpet · 임의공급 경쟁률

getAptLttotPblancScore · APT 당첨가점(평균/최저=커트라인/최고) — APT 전용, 다른 유형은 가점제 대상이 아니라 이 데이터 자체가 없음

getAPTSpsplyReqstStus · APT 특별공급 신청현황(유형×지역별 접수건수/배정세대수) — APT 전용

④ 코드 파일 위치

`backend/app/data_pipeline/competition_client.py` 한 파일에서 위 7개 엔드포인트를 전부 호출·저장합니다. 터미널에서 `cd backend` 후 `python -m app.data_pipeline.competition_client`로 실행하면 경쟁률 → 당첨가점 → 특별공급 신청현황 순서로 돌면서 `historical_results` 테이블에 저장됩니다. 저장 방식은 `(external_id, unit_type, category_label)` 조합을 기준으로 이미 있으면 덮어쓰고(update) 없으면 새로 추가(insert)하는 **upsert** 방식이라, 같은 명령을 여러 번 실행해도 행이 중복되지 않습니다.